

الموسوعة البرمجية والدليل الشامل لأكواد وصياغات منصة ردود (الإصدار الثاني V2)

Rudood AI Platform V2 – Complete Code, Functions & Syntax Master Reference Manual

الوثيقة: الدليل المرجعي الموسوعي الشامل لكافة الأكواد والملفات والدوال والمتحكمات

الإصدار: Enterprise Decoupled SPA Architecture 2.0

الحزمة التقنية: React 19 (v19.0.0) | TypeScript 5.7 | Vite 6 | Zustand 5 | Axios | Laravel 11 REST API

| PostgreSQL 16 pgvector | PHP 8.4

المستودع الرسمي: <https://github.com/abdo544445/rudood-platform-v2>

فهرس الأقسام التفصيلي (Complete Table of Contents)

- القسم 1: المعايير الهندسية والصياغة الصارمة (Strict Typing & Code Standards)
- القسم 2: التشريح الشامل للواجهة الأمامية (Frontend React 19 SPA - كل ملف ومكون ودالة)
 - 2.1 نقطة الدخول وشجرة التوجيه (main.tsx & App.tsx)
 - 2.2 متجر الحالة العالمي والمصادقة (store/useAuthStore.ts)
 - 2.3 عميل الشبكة واعتراض التوكنز (services/apiClient.ts)
 - 2.4 حارس المسارات المحمية وصلاحيات الأيمن (components/common/ProtectedRoute.tsx)
 - 2.5 لوحة الأوامر السريعة والاختصارات (components/common/CommandPalette.tsx)
 - 2.6 محرك الخلفية الفلكية التفاعلية (components/common/AmbientCanvas.tsx)
 - 2.7 مكونات التخطيط (, AppLayout , Header , Sidebar , PublicNavbar , PublicFooter)
 - 2.8 جناح الإدارة العليا ذو الـ 8 تبويبات بالتفصيل (pages/admin/AdminPage.tsx)
 - 2.9 صفحة إعدادات محرك البوت وجالب النماذج الحي (pages/settings/BotSettingsPage.tsx)
 - 2.10 صفحة قاعدة المعرفة ومستخرج الأسئلة الذكي (pages/knowledge/KnowledgeBasePage.tsx)

- 2.11 صفحة صندوق المحادثات الحي ومحاكي العميل (pages/chat/LiveChatPage.tsx)
- 2.12 صفحة مختبر الذكاء الاصطناعي وقياس السرعة (pages/playground/PlaygroundPage.tsx)
- 2.13 لوحة تحكم التاجر ومؤشرات الأداء (pages/dashboard/DashboardPage.tsx)
- 2.14 الصفحات العامة والمدونة وقارئ المقالات (*/pages/public)

3. القسم 3: التشرية الشامل للواجهة الخلفية (Backend Laravel 11 REST API) 🐘

- 3.1 خريطة مسارات الـ REST API الكاملة (routes/api.php)
- 3.2 وحدة تحكم الإدارة العليا ومحاكي الـ (AdminController.php) SQL
- 3.3 وحدة تحكم إعدادات محرك البوت (BotSettingsController.php)
- 3.4 وحدة تحكم قاعدة المعرفة واستخراج الـ (KnowledgeBaseController.php) FAQ
- 3.5 وحدة تحكم مختبر الذكاء الاصطناعي (PlaygroundController.php)
- 3.6 وحدة تحكم المحادثات والتدخل البشري (ChatController.php)
- 3.7 وحدة تحكم المصادقة وإصدار التوكنز (AuthController.php)
- 3.8 الخدمات وخوارزميات الذكاء الاصطناعي (, RagService , AiService WhatsAppInteractiveService)

4. القسم 4: تشرية استعلامات البحث الدلالي بالمتجهات (pgvector Syntax) 🔍

5. القسم 5: تشرية خادم الويب سوكيت والبث اللحظي (websocket/server.js) 📡

6. القسم 6: تشرية محرك الاختبارات الآلية الشاملة (118 اختباراً ناجحاً بنسبة 100%) 🏆

القسم 1: المعايير الهندسية والصياغة الصارمة 🏛️

تلتزم منصة ردود بأعلى معايير الجودة الصارمة المعمول بها في كبرى المؤسسات التقنية:

- **TypeScript 5.7 Strict Mode**: تفعيل الصرامة الكاملة (strict: true)، مما يمنع استخدام any الضمني ويفرض تحديد أنواع البيانات لجميع الـ Props، الـ States، واستجابات الـ API.
- **Component-Driven Decoupled Design**: كل صفحة ومكون مستقل بذاته، مع فصل كامل لمنطق الاتصال بالشبكة داخل services/apiClient.ts ومنطق الحالة داخل store/useAuthStore.ts.
- **PHP 8.4 Architecture**: استخدام الـ Property Promotion، دوال الـ Match، والأنواع المدمجة الصارمة لكافة الدوال وقيم الإرجاع.

القسم 2: التشرح الشامل للواجهة الأمامية

2.1 نقطة الدخول وشجرة التوجيه (frontend/src/main.tsx & App.tsx)

(أ) ملف الدخول الأساسي main.tsx :

يقوم بربط تطبيق React في الـ Root DOM Element مع تفعيل الـ StrictMode:

```
// frontend/src/main.tsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.tsx';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

(ب) شجرة التوجيه App.tsx :

تحتوي على نظام التوجيه الكامل (BrowserRouter) مقسمة إلى مسارات عامة لا تتطلب تسجيلاً ومسارات خاصة محمية بمكون

: ProtectedRoute

```

// frontend/src/App.tsx (Excerpt)
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { ProtectedRoute } from '../components/common/ProtectedRoute';
import { AppLayout } from '../components/layout/AppLayout';

// Public Pages
import { HomePage } from '../pages/public/HomePage';
import { FeaturesPage } from '../pages/public/FeaturesPage';
import { PricingPage } from '../pages/public/PricingPage';
import { HowItWorksPage } from '../pages/public/HowItWorksPage';
import { DemoPage } from '../pages/public/DemoPage';
import { ContactPage } from '../pages/public/ContactPage';
import { BlogPage } from '../pages/public/BlogPage';
import { LoginPage } from '../pages/auth/LoginPage';
import { RegisterPage } from '../pages/auth/RegisterPage';

// Protected Merchant & Admin Pages
import { DashboardPage } from '../pages/dashboard/DashboardPage';
import { LiveChatPage } from '../pages/chat/LiveChatPage';
import { ChannelsPage } from '../pages/channels/ChannelsPage';
import { KnowledgeBasePage } from '../pages/knowledge/KnowledgeBasePage';
import { PlaygroundPage } from '../pages/playground/PlaygroundPage';
import { BotSettingsPage } from '../pages/settings/BotSettingsPage';
import { AdminPage } from '../pages/admin/AdminPage';

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Public Routes */}
        <Route path="/" element={<HomePage />} />
        <Route path="/features" element={<FeaturesPage />} />
        <Route path="/pricing" element={<PricingPage />} />
        <Route path="/how-it-works" element={<HowItWorksPage />} />
        <Route path="/demo" element={<DemoPage />} />
        <Route path="/contact" element={<ContactPage />} />
        <Route path="/blog" element={<BlogPage />} />
        <Route path="/login" element={<LoginPage />} />
        <Route path="/register" element={<RegisterPage />} />

        {/* Protected Merchant Routes */}
        <Route path="/dashboard" element={<ProtectedRoute><AppLayout><DashboardPage /></ProtectedRoute>} />
        <Route path="/chat" element={<ProtectedRoute><AppLayout><LiveChatPage /></AppLayout>} />
      </Routes>
    </BrowserRouter>
  );
}

```

```

<Route path="/channels" element={<ProtectedRoute><AppLayout><ChannelsPage /></A
<Route path="/knowledge" element={<ProtectedRoute><AppLayout><KnowledgeBasePage
<Route path="/playground" element={<ProtectedRoute><AppLayout><PlaygroundPage /
<Route path="/settings" element={<ProtectedRoute><AppLayout><BotSettingsPage />

{/* Protected Super Admin Route */}
<Route path="/admin" element={<ProtectedRoute requireAdmin={true}><AppLayout><A

{/* Fallback */}
<Route path="*" element={<Navigate to="/" replace /> } />
</Routes>
</BrowserRouter>
);
}

```

2.2 متجر الحالة العالمي والمصادقة (frontend/src/store/useAuthStore.ts)

يدير بيانات المستخدم المسجل، رمز التوكن (JWT/Bearer Token)، صلاحيات الوصول، وميزة انتحال المتجر (Impersonation) مع الحفظ التلقائي في localStorage :

```

// frontend/src/store/useAuthStore.ts
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

export interface User {
  id: number;
  name: string;
  email: string;
  role: 'admin' | 'owner' | 'agent';
  store_id?: number;
  store_name?: string;
}

interface AuthState {
  token: string | null;
  user: User | null;
  isAuthenticated: boolean;
  login: (token: string, user: User) => void;
  logout: () => void;
  impersonate: (token: string, user: User) => void;
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      token: null,
      user: null,
      isAuthenticated: false,
      login: (token, user) => set({ token, user, isAuthenticated: true }),
      logout: () => {
        localStorage.removeItem('rudood-token');
        set({ token: null, user: null, isAuthenticated: false });
      },
      impersonate: (token, user) => set({ token, user, isAuthenticated: true }),
    }),
    {
      name: 'rudood-auth-storage',
    }
  )
);

```

2.3 عميل الشبكة واعتراض التوكنز (frontend/src/services/apiClient.ts)

يقوم بتهيئة مكتبة **Axios** مع:

1. **Request Interceptor**: استخراج التوكن من Zustand وحفظه تلقائياً في ترويسة

. <Authorization: Bearer <token

2. **Response Interceptor**: التقاط أخطاء انتهاء الجلسة 401 Unauthorized وتوجيه الحالة وتوجيه المستخدم لصفحة الدخول.

```
// frontend/src/services/apiClient.ts
import axios from 'axios';
import { useAuthStore } from '../store/useAuthStore';

export const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8000/api/v1',
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  },
});

// Auto-inject Bearer Token
apiClient.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Global 401 Session Expired Handler
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      useAuthStore.getState().logout();
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

2.4 حارس المسارات المحمية

(frontend/src/components/common/ProtectedRoute.tsx)

يمنع الزوار غير المسجلين من الدخول للمسارات الخاصة، كما يمنع التجار من دخول جناح المدير الأعلى : admin/

```
// frontend/src/components/common/ProtectedRoute.tsx
import React from 'react';
import { Navigate } from 'react-router-dom';
import { useAuthStore } from '../../store/useAuthStore';

interface ProtectedRouteProps {
  children: React.ReactNode;
  requireAdmin?: boolean;
}

export const ProtectedRoute: React.FC<ProtectedRouteProps> = ({ children, requireAdmin
  const { isAuthenticated, user } = useAuthStore();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  if (requireAdmin && user?.role !== 'admin') {
    return <Navigate to="/dashboard" replace />;
  }

  return <>{children}</>;
};
```

2.5 محرك الخلفية الفلكية التفاعلية

(frontend/src/components/common/AmbientCanvas.tsx)

محرك رسم مبني على HTML5 Canvas يحاكي فيزياء حركة النجوم والجسيمات في الفضاء وتفاعلها مع مؤشر الفأرة:


```
// frontend/src/components/common/AmbientCanvas.tsx (Excerpt)
import React, { useEffect, useRef } from 'react';

export const AmbientCanvas: React.FC = () => {
  const canvasRef = useRef<HTMLCanvasElement | null>(null);

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    const ctx = canvas.getContext('2d');
    if (!ctx) return;

    let animationFrameId: number;
    let width = (canvas.width = window.innerWidth);
    let height = (canvas.height = window.innerHeight);

    // Generate 70 floating luxury particles
    const particles = Array.from({ length: 70 }, () => ({
      x: Math.random() * width,
      y: Math.random() * height,
      radius: Math.random() * 1.8 + 0.5,
      vx: (Math.random() - 0.5) * 0.4,
      vy: (Math.random() - 0.5) * 0.4,
      alpha: Math.random() * 0.6 + 0.2,
    }));

    const render = () => {
      ctx.clearRect(0, 0, width, height);
      particles.forEach((p) => {
        p.x += p.vx;
        p.y += p.vy;
        if (p.x < 0) p.x = width;
        if (p.x > width) p.x = 0;
        if (p.y < 0) p.y = height;
        if (p.y > height) p.y = 0;

        ctx.beginPath();
        ctx.arc(p.x, p.y, p.radius, 0, Math.PI * 2);
        ctx.fillStyle = `rgba(212, 175, 55, ${p.alpha})`; // Gold highlight
        ctx.fill();
      });
      animationFrameId = requestAnimationFrame(render);
    };
  });
};
```

```

render();
return () => cancelAnimationFrame(animationFrameId);
}, []);

return <canvas ref={canvasRef} className="fixed inset-0 pointer-events-none z-0" />;
};

```

2.6 جناح الإدارة العليا ذو الـ 8 تبويبات (frontend/src/pages/admin/AdminPage.tsx)

يعد هذا الملف أكبر مجمع إداري على المنصة، حيث يحتوي على 8 موديولات تشغيلية متكاملة:

1. **Overview (النظرة العامة):** عرض بطاقات الـ KPIs الرئيسية (المتاجر، الرسائل، معدل الأتمتة، المستخدمين النشطين)، رسم بياني لسرعة الاستجابة، وتوزيع مزودي الذكاء الاصطناعي.
2. **Advanced Analytics (التحليلات المالية والمتقدمة):** حساب الإيراد الشهري والسنوي (MRR و ARR)، ومعدل استقرار طوابير Redis، مع زر مخصص لتنظيف المهام الفاشلة (Prune Failed Jobs).
3. **Workspaces (إدارة الشركات والمتاجر):** جدول تفاعلي للمتاجر مع الفلترة حسب الحالة، نافذة منبثقة لإنشاء متجر جديد مع بيانات المالك، وزر تسجيل الدخول كمالك المتجر بنقرة واحدة (Click Impersonation-1).
4. **Users Directory (دليل المستخدمين):** عرض جميع حسابات النظام مع إمكانية ترقية أو تغيير الصلاحيات (Admin, Owner, Agent) وحذف الحسابات.
5. **Blog CMS (نظام المقالات):** إدارة مقالات المدونة مع نافذة إنشاء وتعديل المقال وتحديد الكلمات المفتاحية ونشره فوراً.
6. **Database Explorer (مستكشف قواعد البيانات ومحكي الـ SQL):** فحص الجداول وعدد السجلات وحجم قاعدة البيانات، مع طرفية SQL للقراءة فقط لتنفيذ الاستعلامات السريعة وعرضها في جدول تفاعلي.
7. **Audit Trail (سجل التدقيق الأمني المؤسسي):** جدول زمني لكافة العمليات الحساسة مع رقم الآي بي (IP Address) وتوقيت التنفيذ ونوع العملية.
8. **System Infrastructure (البنية التحتية للنظام):** تفعيل أو إلغاء وضع الصيانة بضغط زر، وزر لتفريغ الكاش ومراقبة استهلاك الذاكرة.

```
// frontend/src/pages/admin/AdminPage.tsx (Core Impersonate & SQL Runner Logic)
const handleImpersonate = async (workspaceId: number) => {
  try {
    const res = await apiClient.post(`/admin/workspaces/${workspaceId}/impersonate`);
    impersonate(res.data.token, res.data.user);
    navigate('/dashboard');
  } catch (err: any) {
    alert(err.response?.data?.message || 'فشل انتحال جلسة المتجر');
  }
};

const handleRunSql = async () => {
  if (!sqlQuery.trim()) return;
  setSqlLoading(true);
  try {
    const res = await apiClient.post('/admin/database/query', { query: sqlQuery });
    setSqlResult(res.data.rows);
    setSqlError(null);
  } catch (err: any) {
    setSqlError(err.response?.data?.message || 'فشل تنفيذ استعلام SQL');
  } finally {
    setSqlLoading(false);
  }
};
```

2.7 صفحة إعدادات محرك البوت (frontend/src/pages/settings/BotSettingsPage.tsx)

تمكن التاجر من التحكم الدقيق في محرك الذكاء الاصطناعي:

- **Master Active Switch**: مفتاح تشغيل/إيقاف ردود البوت للمتجر بأكمله.
- **AI Provider Selector**: التبديل بين Gemini و OpenAI و Claude مع حقل مخصص لـ Base URL للنماذج المتوافقة.
- **Live Model Fetcher**: زر لجلب الموديلات المتاحة حياً من الـ API المعتمد.
- **Sliders**: شرائح ضبط دقيقة لدرجة حرارة الإجابة (Creativity: 0.0 إلى 1.0) والحد الأقصى للتوكنز.

```
// frontend/src/pages/settings/BotSettingsPage.tsx (Snippet)
const fetchLiveModels = async () => {
  setModelLoading(true);
  try {
    const res = await apiClient.get('/bot/models', { params: { provider: settings.ai_pr
    setAvailableModels(res.data.models);
  } catch (err) {
    console.error('Failed to fetch models', err);
  } finally {
    setModelLoading(false);
  }
};
```

2.8 صفحة صندوق المحادثات والتدخل البشري (frontend/src/pages/chat/LiveChatPage.tsx)

تمكن الوكيل البشري من متابعة المحادثات لحظياً:

- قائمة المحادثات النشطة المصنفة حسب القنوات.
- زر التدخل البشري (Human Takeover): إيقاف الذكاء الاصطناعي لمحادثة معينة فوراً بطلب . api/v1/conversations/{id}/toggle-bot/
- محاكي محادثات مدمج (Conversation Simulator): إرسال رسائل تجريبية كعميل وتلقي رد البوت واختبار سرعة الاستجابة.

🐘 القسم 3: التشرح الشامل للواجهة الخلفية (Laravel REST API 11)

3.1 خريطة مسارات الـ REST API الكاملة (backend/routes/api.php)

توضح كافة المسارات المتاحة وصلاحيات الوصول لكل مسار:

```

// backend/routes/api.php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\AdminController;
use App\Http\Controllers\Api\BotSettingsController;
use App\Http\Controllers\Api\ChatController;
use App\Http\Controllers\Api\KnowledgeBaseController;
use App\Http\Controllers\Api\PlaygroundController;
use App\Http\Controllers\Api\PublicController;

Route::prefix('v1')->group(function () {
    // 1. Public & Authentication
    Route::post('/auth/login', [AuthController::class, 'login']);
    Route::post('/auth/register', [AuthController::class, 'register']);
    Route::get('/public/stats', [PublicController::class, 'stats']);
    Route::get('/blog/articles', [PublicController::class, 'articles']);
    Route::get('/blog/articles/{slug}', [PublicController::class, 'articleBySlug']);
    Route::post('/contact', [PublicController::class, 'storeContact']);

    // 2. Protected Merchant Endpoints
    Route::middleware('auth:sanctum')->group(function () {
        Route::get('/dashboard/stats', [ChatController::class, 'dashboardStats']);
        Route::get('/conversations', [ChatController::class, 'index']);
        Route::get('/conversations/{id}/messages', [ChatController::class, 'messages']);
        Route::post('/conversations/{id}/messages', [ChatController::class, 'sendMessage']);
        Route::post('/conversations/{id}/toggle-bot', [ChatController::class, 'toggleBot']);

        Route::get('/bot/settings', [BotSettingsController::class, 'show']);
        Route::put('/bot/settings', [BotSettingsController::class, 'update']);
        Route::get('/bot/models', [BotSettingsController::class, 'fetchModels']);

        Route::get('/knowledge-base', [KnowledgeBaseController::class, 'index']);
        Route::post('/knowledge-base/upload', [KnowledgeBaseController::class, 'upload']);
        Route::post('/knowledge-base/faq/{id}', [KnowledgeBaseController::class, 'extra']);

        Route::post('/playground/test', [PlaygroundController::class, 'testPrompt']);
    });

    // 3. Super Admin Command Suite
    Route::middleware(['auth:sanctum', 'role:admin'])->prefix('admin')->group(function () {
        Route::get('/overview', [AdminController::class, 'overview']);
        Route::get('/analytics', [AdminController::class, 'analytics']);
        Route::post('/analytics/prune-failed-jobs', [AdminController::class, 'pruneFail

```

```

Route::get('/workspaces', [AdminController::class, 'workspaces']);
Route::post('/workspaces', [AdminController::class, 'createWorkspace']);
Route::post('/workspaces/{id}/impersonate', [AdminController::class, 'impersonate']);
Route::patch('/workspaces/{id}/status', [AdminController::class, 'updateWorkspaceStatus']);
Route::get('/users', [AdminController::class, 'users']);
Route::patch('/users/{id}/role', [AdminController::class, 'updateUserRole']);
Route::delete('/users/{id}', [AdminController::class, 'deleteUser']);
Route::get('/articles', [AdminController::class, 'articles']);
Route::post('/articles', [AdminController::class, 'createArticle']);
Route::put('/articles/{id}', [AdminController::class, 'updateArticle']);
Route::delete('/articles/{id}', [AdminController::class, 'deleteArticle']);
Route::get('/database/tables', [AdminController::class, 'databaseTables']);
Route::post('/database/query', [AdminController::class, 'runSqlTerminal']);
Route::get('/audit-trail', [AdminController::class, 'auditTrail']);
Route::post('/system/maintenance', [AdminController::class, 'toggleMaintenance']);
Route::post('/system/clear-cache', [AdminController::class, 'clearCache']);
});
});

```

3.2 كود استخراج الأسئلة الشائعة آلياً بالذكاء الاصطناعي (KnowledgeBaseController::extractFaq)

تقرأ الدالة المستند المخزن، وتمرره للـ LLM لتوليد أسئلة وأجوبة محددة وحفظها تلقائياً:

```
// backend/app/Http/Controllers/Api/KnowledgeBaseController.php
public function extractFaq(int $id): JsonResponse
{
    $document = KnowledgeDocument::where('store_id', auth()->user()->store_id)
        ->findOrFail($id);

    $prompt = "JSOI قم بتحليل المستند التالي واستخرج منه أهم 5 أسئلة وأجوبة شائعة بتنسيق"

    $response = $this->aiService->generateResponse($prompt, [
        'temperature' => 0.2,
        'format' => 'json'
    ]);

    $faqs = json_decode($response, true) ?? [];

    foreach ($faqs as $item) {
        AutoRule::create([
            'store_id' => $document->store_id,
            'keyword' => $item['question'],
            'response' => $item['answer'],
            'is_active' => true,
        ]);
    }

    return response()->json([
        'success' => true,
        'message' => 'تم استخراج وتوليد الأسئلة الشائعة بنجاح',
        'faqs' => $faqs
    ]);
}
```

القسم 4: تشرح استعلامات البحث الدلالي بالمتجهات (pgvector)

تستخدم المنصة معامل المسافة المتجهية <=> (Cosine Distance) لاكتشاف المقاطع النصية الأقرب دلاليًا لسؤال العميل:

```
// pgvector Cosine Distance Query in PostgreSQL
$queryEmbedding = $this->embeddingService->getVector($customerQuestion);

$relevantChunks = DB::table('knowledge_chunks')
    ->join('knowledge_documents', 'knowledge_chunks.document_id', '=', 'knowledge_docum
    ->where('knowledge_documents.store_id', $storeId)
    ->where('knowledge_documents.is_active', true)
    ->select('knowledge_chunks.content', 'knowledge_chunks.title')
    ->selectRaw('1 - (knowledge_chunks.embedding <=> ?::vector) AS similarity', [json_e
    ->whereRaw('1 - (knowledge_chunks.embedding <=> ?::vector) >= ?', [json_encode($que
    ->orderByDesc('similarity')
    ->limit(4)
    ->get();
```

القسم 5: تشرح خادم الويب سوكت والبث اللحظي 📡

خادم Node.js مستقل يستمع لأحداث Redis Pub/Sub ويبثها عبر [Socket.IO](https://socket.io/) إلى الغرف المعزولة برقم المتجر (<store_id >):


```
// backend/websocket/server.js
const { createServer } = require('http');
const { Server } = require('socket.io');
const Redis = require('ioredis');

const httpServer = createServer();
const io = new Server(httpServer, {
  cors: { origin: '*', methods: ['GET', 'POST'] }
});

const redis = new Redis(process.env.REDIS_URL || 'redis://127.0.0.1:6379');

redis.subscribe('rudood-realtime', (err, count) => {
  if (err) console.error('Redis subscribe error:', err);
});

redis.on('message', (channel, message) => {
  const data = JSON.parse(message);
  if (data.store_id) {
    io.to(`store_${data.store_id}`).emit(data.event, data.payload);
  }
});

io.on('connection', (socket) => {
  socket.on('join_store', (storeId) => {
    socket.join(`store_${storeId}`);
  });
});

httpServer.listen(3000, () => {
  console.log('🚀 Rudood WebSocket Server active on port 3000');
});
```

القسم 6: تشرح محرك الاختبارات الآلية الشاملة

يتم تنفيذ 118 فحصاً برمجياً دقيقاً في ثوانٍ معدودة للتحقق من تكامل كافة الطبقات:

```
cd backend && php tests_suite_runner.php
```

=====

RUDOOD AI PLATFORM – COMPREHENSIVE AUTOMATED TEST SUITE (V2 DECOUPLED)

=====

[01] Token Authentication & Session Guards:	14 Tests [PASSED]
[02] Multi-Channel Webhook Ingestion Pipeline:	22 Tests [PASSED]
[03] Semantic RAG & pgvector Similarity Search:	18 Tests [PASSED]
[04] Omni-Channel Live Chat & Human Takeover:	16 Tests [PASSED]
[05] Super Admin 8-Module Command Suite & SQL Runner:	28 Tests [PASSED]
[06] Public Portal, Blog CMS & Inquiries Router:	20 Tests [PASSED]

TOTAL TESTS: 118 / 118 (100% SUCCESS) | DURATION: 2.14s

=====

منصة ردود للذكاء الاصطناعي – الإصدار الثاني (V2)

المستودع الرسمي: <https://github.com/abdo544445/rudood-platform-v2>